Cerinte Proiect - Magazin Online Full-Stack (Node.js, Express, Prisma + React)

Punctaj si reguli generale (Total: 10 puncte)

Proiectul porneste de la **bazele de backend si frontend furnizate** (Seminar 2 si Seminar 3), peste care veti implementa si integra cele **3 module functionale** de mai jos (atat pe server, cat si in clientul React).

Componenta	Punctaj	Detalii
Integrare baza proiect	**1p**	Rularea corecta a proiectelor de backend si frontend furnizate si integrarea noilor rute/pagini in aplicatie (`App.jsx`, `Navbar.jsx`)
Modulul A: Recenzii produse	**3p**	Model `Review` + rute API (1.5p) + afisare, adaugare si stergere recenzii in React (1.5p)
Modulul B: Cos persistent	**3p**	Model `CartItem` + rute API (1.5p) + sincronizare cos `localStorage` / baza de date in React (1.5p)
Modulul C: Panou Admin	**3p**	Rute protejate `ADMIN` (1p) + pagini de administrare produse CRUD si comenzi (2p)
TOTAL	**10p**	

> **Recomandare de lucru:** Implementati mai intai modelele noi in `prisma/schema.prisma` (`Review` si `CartItem`), rulati migrarea (`npx prisma migrate dev`) si testati rutele noi in Postman/Thunder Client, apoi construiti componentele si paginile in React.

A. Recenzii pentru produse (3p)

Utilizatorii autentificati pot lasa recenzii produselor, iar orice vizitator poate vedea notele si comentariile.

```
User  1 ——— N  Review  N ——— 1  Product
```

Adaugati modelul in `schema.prisma`:

```
model Review {
    id        Int      @id @default(autoincrement())
    rating    Int
    comment   String?
    createdAt DateTime @default(now())

    userId    Int
    productId Int
    user      User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
    product   Product  @relation(fields: [productId], references: [id], onDelete:
```

```
  Cascade)

      @@unique([userId, productId])
  }
```

Adaugati `reviews Review[]` in modelele `User` si `Product`, apoi rulati migrarea.

`@@unique([userId, productId])` interzice mai multe recenzii ale aceluiasi utilizator pentru acelasi produs.

Cerinte Backend (1.5p)

1. Creati migrarea pentru modelul `Review`.
2. `POST /api/products/:id/reviews` (utilizator autentificat): creeaza o recenzie. Validati `rating` (intreg intre 1 si 5). Daca utilizatorul a recenzat deja produsul, returnati `409`.
3. `GET /api/products/:id/reviews` (public): returneaza recenziile produsului, cu numele autorului (`include` + `select`, **fara** email sau parola), ordonate descrescator dupa data.
4. Modificati `GET /api/products/:id` astfel incat raspunsul sa contina si `averageRating` si `reviewCount`.
5. `DELETE /api/reviews/:id`: o recenzie poate fi stearsa doar de autorul ei sau de `ADMIN` (altfel `403`).

Cerinte Frontend - React (1.5p)

1. Pe `ProductDetailPage.jsx`, afisati media notelor (`averageRating`) si numarul de recenzii (`reviewCount`) langa titlul produsului.
2. Sub detaliile produsului, afisati lista recenziilor (`GET /api/products/:id/reviews`) cu numele autorului, rating-ul (ex. `4 / 5`), comentariul si data.
3. Daca utilizatorul este autentificat, afisati un formular de adaugare recenzie (`select 1-5` + `textarea` pentru comentariu) conectat la `POST /api/products/:id/reviews`.
4. Afisati butonul **Sterge** (`DELETE /api/reviews/:id`) doar in dreptul recenziilor scrise de utilizatorul curent (`review.userId === user?.id`) sau daca utilizatorul are rolul `ADMIN`.

B. Cos de cumparaturi persistent (3p)

Cosul de cumparaturi ramane salvat chiar daca utilizatorul inchide browserul sau se conecteaza de pe alt dispozitiv.

```
User  1 ——— N  CartItem  N ——— 1  Product
```

Adaugati modelul in `schema.prisma`:

```
model CartItem {
    id        Int      @id @default(autoincrement())
    quantity  Int
    createdAt DateTime @default(now())

    userId    Int
    productId Int
    user      User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
    product   Product  @relation(fields: [productId], references: [id], onDelete:
Cascade)

    @@unique([userId, productId])
}
```

Adaugati `cartItems CartItem[]` in `User` si `Product`, apoi rulati migrarea.

Cerinte Backend (1.5p)

1. `GET /api/cart` (utilizator autentificat): returneaza liniile din cosul utilizatorului curent cu detaliile produsului (`id`, `name`, `price`, `stock`, `imageUrl`).
2. `POST /api/cart` (utilizator autentificat): primeste `{ productId, quantity }`. Daca produsul exista deja in cos, aduna cantitatea (fara a depasi `product.stock`, altfel `409`); daca nu exista, il adauga.
3. `PATCH /api/cart/:productId` (utilizator autentificat): actualizeaza cantitatea unui produs din cos (validare `quantity >= 1` si `quantity <= product.stock`).
4. `DELETE /api/cart/:productId` (utilizator autentificat): elimina un produs din cos.
5. Modificati `POST /api/orders` astfel incat comanda sa fie creata automat din `CartItem`-urile utilizatorului si nu din `orderLines` primite din API asa cum se intampla in `Seminar 2`, iar la finalul tranzactiei cosul din baza de date sa fie golit (`deleteMany`).

Cerinte Frontend - React (1.5p)

1. In `CartContext.jsx`, sincronizati starea cosului cu API-ul (`GET /api/cart`) ori de cate ori `user` devine autentificat.
2. Pentru vizitatorii neautentificati, pastrati cosul in `localStorage` (`localStorage.getItem("cart")` / `localStorage.setItem("cart", ...)`), iar la login sincronizati produsele locale cu serverul.

3. Actualizati `CartPage.jsx` astfel incat modificarile de cantitate (`+` / `-`), stergerea produselor si plasarea comenzii sa apeleze rutele `/api/cart` si `/api/orders`.

C. Panou de administrare pentru produse si comenzi (3p)

Administratorii (`role === "ADMIN"`) pot gestiona catalogul de produse si statusul comenzilor direct din interfata web.

Cerinte Backend (1p)

1. Verificati si completati validarea datelor pe rutele `POST /api/products`, `PATCH /api/products/:id` si `DELETE /api/products/:id` (protejate cu `authenticate, requireRole("ADMIN")`): `name` nevid, `price > 0`, `category` nevida, `stock >= 0`.
2. Asigurati-va ca rutele `GET /api/admin/orders` (lista tuturor comenzilor cu `name` si `email` ale clientului) si `PATCH /api/admin/orders/:id/status` sunt functionale.

Cerinte Frontend - React (2p)

1. Creati componenta `AdminRoute.jsx` care verifica `user?.role === "ADMIN"` (in caz contrar, redirectioneaza catre `/products`).
2. Afisati in `Navbar.jsx` link-uri catre panoul de administrare vizibile doar cand `user?.role === "ADMIN"`.
3. Creati pagina `/admin/products` (`AdminProductsPage.jsx`):
    - formular pentru adaugarea unui produs nou (`POST /api/products`) si editarea unui produs existent (`PATCH /api/products/:id`);
    - tabel cu produsele existente, pretul, categoria, stocul curent si buton de stergere (`DELETE /api/products/:id`) cu confirmare (`window.confirm`).
4. Creati pagina `/admin/orders` (`AdminOrdersPage.jsx`):
    - tabel cu toate comenzile din sistem (`GET /api/admin/orders`), afisand clientul (`name`, `email`), produsele, totalul si data;
    - un `<select>` pe fiecare rand care permite schimbarea statusului (`PENDING`, `PAID`, `SHIPPED`, `DELIVERED`, `CANCELLED`) prin `PATCH /api/admin/orders/:id/status`.